FIAP - 2TIAOA | PROJETO CARDIOIA - FASE 7 (MVP FINAL)

Cap 1: Da Construção de Módulos à Solução Integrada - Relatório Técnico & Arquitetura

Integrantes do Grupo: Davi Santos Ferreira & Lais Kurahashi | Turma: 2TIAOA - 2026

Disciplina / Etapa: Fase 7 - MVP Final: Da Construção de Módulos à Solução Integrada

1. Links Oficiais da Solução Integrada (Clicáveis)

1. Aplicação Web Publicada (Vercel SPA com CI/CD ativo):
https://fiap-project-cardio-ia-delta.vercel.app

2. Aplicativo Mobile Android (Download do .APK via Expo EAS Build):
https://expo.dev/artifacts/eas/5-zVREEnJZ6c7p0Ryhjkn65hlqBnluC4IfWS9qX_5JI.apk

3. Simulação de Hardware IoT em MicroPython (Ambiente Wokwi ESP32):
https://wokwi.com/projects/463793755500080129

4. Repositório Oficial no GitHub (Código-Fonte Integrado e Documentos):
https://github.com/Raquerd/FIAP_Project_CardioIA

2. Resumo Executivo da Solução Integrada (Visão Fim-a-Fim)

O projeto CardioIA atinge sua maturidade técnica consolidando todos os módulos desenvolvidos ao longo do curso em um ecossistema hospitalar coeso, orquestrado por Python e disponível em produção para múltiplos canais de atendimento (Web e Mobile). O fluxo contempla a captura de sinais vitais de borda simulados em MicroPython (ESP32 no Wokwi), a transmissão resiliente de telemetria via MQTT/HTTP para um Backend Integrador em Python hospedado na nuvem (AWS Elastic Beanstalk), a inferência contínua por motores de Inteligência Artificial supervisionada (Classificação de ECG por CNN/ResNet50 da Fase 4, Assistente Conversacional Watson/LLM da Fase 5 e Predição de Crises por Random Forest e Sistema Multiagente da Fase 6) e a entrega de indicadores clínicos e laudos em tempo real nas interfaces do paciente e da equipe médica.

3. Divisão de Responsabilidades na Fase 7

Parte 1 - Deploy e Distribuição Profissional Front-End & Mobile (Lais Kurahashi):

- Publicação da aplicação Web em React + Vite na Vercel com suporte a rotas SPA (web/vercel.json) e CI/CD ativo.
- Configuração de build mobile Expo EAS (mobile/app.json com domínio invertido e mobile/eas.json perfil preview).
- Geração bem-sucedida do pacote instalável .apk Android no dashboard da Expo e documentação no README.md.
- Refinamento de UX nas telas de login, visão geral, envio de imagens de ECG e visualização de probabilidades.

Parte 2 - Integração do Ecossistema, MicroPython & Arquitetura Final (Davi Ferreira):

- Transição de C/C++ para MicroPython no ESP32 (iot/main.py), conectando DHT22, Potenciômetro ADC, OLED e LED.
- Implementação do Buffer de Borda (Edge Computing) com tolerância a falhas de rede e sincronização em lote.
- Construção do Backend Integrador em Python (backend/app.py e backend/integrador_service.py) unificando IA e IoT.
- Criação de rota e interface de Telemetria & Risco em tempo real integrada à Web e elaboração do Relatório Técnico.

1. Transição de Hardware: C/C++ para MicroPython no ESP32

Na Fase 3, o protótipo de sensoriamento havia sido desenvolvido em C++ para a IDE Arduino. Na presente fase, conforme a diretriz de modernização tecnológica, realizamos a transição completa da lógica de firmware para MicroPython (firmware v1.20+ no ESP32 DevKit v4). Essa abordagem eleva a manutenibilidade do código, padroniza toda a pilha tecnológica da CardioIA em Python e permite validação direta de objetos JSON em memória.

Mapeamento Físico de Periféricos e Pinos (ESP32 DevKit v4):

Periferico / Sensor	Pino GPIO	Protocolo / Barramento	Funcao Clinica no CardioIA
Sensor DHT22	GPIO 15	Digital 1-Wire	Captura continua de Temperatura Corporal e Umidade
Potenciometro ADC	GPIO 34	Analogico (ADC 11dB)	Simulacao de Frequencia Cardiaca (40 a 200 BPM)
Display OLED SSD1306	GPIO 21 (SDA) GPIO 22 (SCL)	I2C (0x3C, 128x64 px)	Feedback visual local de BPM, Temperatura, Status e Rede
LED Vermelho Alerta	GPIO 2	Digital Output (PWM)	Acionamento fisico instantaneo em Taquicardia ou Febre
Conectividade Wi-Fi	Interno	802.11 b/g/n (STA)	Transmissao MQTT (HiveMQ) e fallback HTTP REST

2. Inteligência de Borda (Edge Computing) e Tolerância a Falhas

A camada IoT não opera como mera transmissora passiva de dados; ela desempenha triagem primária local (Edge Analytics) antes mesmo de acionar a nuvem, garantindo segurança ao paciente em situações críticas:

- Triagem Imediata por Hardware: Ao detectar batimentos acima de 120 BPM ou temperatura superior a 38,0 °C, o microcontrolador aciona o LED vermelho no GPIO 2 e atualiza a tela OLED para '! ALERTA !' em tempo de máquina (<5ms).
- Buffer Circular de Borda (Resiliência Offline-First): Implementamos uma fila circular em memória RAM local com capacidade para 20 registros. Caso a rede Wi-Fi oscile ou o broker MQTT fique inacessível, os sinais vitais são persistidos localmente sem perda de histórico clínico.
- Sincronização Automática em Lote: Quando a conectividade é restabelecida, a rotina sincronizar_buffer() descarrega todos os pacotes acumulados de forma sequencial com carimbo de tempo, limpando o buffer de borda ordenadamente.
- Feedback Visual Completo em OLED SSD1306: Utilizando o driver oficial framebuf ssd1306.py, a interface exibe o cabeçalho do CardioIA, valores instantâneos de BPM e Temperatura, estado clínico e contador de pacotes pendentes.

Estrutura do Pacote de Telemetria Emitido pelo ESP32 (JSON):

```
{"dispositivo_id": "esp32-cardioia-01", "timestamp_ms": 17943518, "bpm": 138, "temperatura": 38.6,
 "umidade": 60.0, "status_clinico": "CRITICO", "alerta_critico": true,
 "edge_decision": "Acionamento de triagem hospitalar"}
```

1. O Núcleo Integrador em Python (backend/app.py)

O núcleo do ecossistema é composto pelo serviço integrador em Python (backend/app.py e backend/integrador_service.py). Ele atua como o orquestrador central de mensagens, recebendo as conexões das aplicações cliente (Vercel e APK) e dos sensores de campo (MicroPython), encaminhando os dados para os motores de Inteligência Artificial especializados.

Conexão Unificada com os Motores de Inteligência Artificial:

Fase 4 - Visão Computacional (CNN do Zero & ResNet50 Transfer Learning):

O endpoint POST /api/ecg/analisar recebe exames de eletrocardiograma em formato PNG/JPEG, pré-processa as imagens em tensores 3x224x224 via PyTorch/Torchvision e infere a probabilidade de classe Normal (N) ou Anomalia (M), retornando o percentual de certeza consumido pelo Web e Mobile.

Fase 5 - Assistente Conversacional & Triagem Clínica (IBM Watson Assistant v2):

Os endpoints /api/session e /api/message gerenciam sessões conversacionais com transbordo inteligente, processando relatos em linguagem natural do paciente e orientando protocolos de preparo de exames com fallback.

Fase 6 - Modelo Preditivo de Risco Hospitalar (Random Forest Classifier):

Alimentado diretamente pelos sinais de telemetria dos sensores (FC média e Temperatura) cruzados com a carga hospitalar. Executa a predição probabilística da base serializada (modelo_risco_cardiaco.joblib), atingindo 99,90% de acurácia com Brier Score de 0,0058 e previsão de pico de risco (0 ou 1).

Fase 6 - Sistema Multiagente Autônomo & Protocolos (OpenAI Agents SDK):

Orquestra 4 agentes especializados (Orquestrador, Analista de Risco, Especialista em Protocolos e Planejador de Resposta) conectados por 3 handoffs explícitos, avaliando as 10 diretrizes clínicas de protocolos_emergencia.json e gerando laudo justificado validado via Pydantic.

Catálogo de Endpoints da API REST Unificada (CORS Habilitado):

Metodo	Rota da API	Finalidade Tecnica	Origem / Consumidor
GET	`/api/status`	Health Check unificado e diagnostico de modulos	Frontend Web, Mobile e Monitoramento
POST	`/api/telemetria`	Ingestao de sinais do ESP32 e predicao de risco	Firmware MicroPython (Wokwi / Hardware)
GET	`/api/telemetria/atual`	Consulta o ultimo sinal vital e risco clinico	Interface Web (Aba Telemetria) e Mobile
GET	`/api/telemetria/historico`	Historico recente para graficos de tendencia	Painel de Acompanhamento Medico
POST	`/api/predicao/risco`	Inferencia preditiva com parametros customizados	Sistema Multiagente e Simulador
POST	`/api/ecg/analisar`	Classificacao de ECG com CNN e ResNet50	Aplicacao Web Vercel e App Mobile APK
POST	`/api/message`	Interacao conversacional com Watson Assistant	Chatbot do Paciente e Triagem Digital

1. Diagrama Técnico da Arquitetura Final CardioIA

O desenho arquitetural abaixo consolida as 5 camadas da solução integrada, evidenciando o percurso completo dos dados desde o sensoriamento físico de borda até as interfaces interativas do usuário final:

CAMADA 1: SENSORIAMENTO & EDGE COMPUTING (MICROPYTHON)
ESP32 DevKit v4 | Sensor DHT22 (Temp) | Potenciometro (BPM 40-200) | OLED SSD1306 | LED Alerta
Logica de borda: Triagem <5ms, buffer circular de resiliencia (20 registros) e reconexao automatica

↓

CAMADA 2: CONECTIVIDADE & TRANSPORTE DE DADOS
Protocolos: MQTT (Broker HiveMQ - porta 1883) com topicos de telemetria e alertas criticos
Fallback: HTTP/1.1 REST JSON multipart direto para o gateway na nuvem via Wi-Fi 802.11b/g/n

↓

CAMADA 3: BACKEND INTEGRADOR PYTHON (NUCLEO DO SISTEMA)
Flask REST API (backend/app.py) | Gunicorn WSGI | Hospedado na AWS Elastic Beanstalk (us-east-2)
Servico Integrador: Ingestao em tempo real, CORS, historico em memoria e orquestracao multi-modelo

↓

CAMADA 4: MOTORES DE INTELIGÊNCIA ARTIFICIAL (IA HOSPITALAR)
1. Visao Computacional: CNN e ResNet50 (PyTorch) para laudo de imagens de eletrocardiograma (Fase 4)
2. Assistente Conversacional: IBM Watson Assistant v2 (NLP de triagem e agendamentos) (Fase 5)
3. Modelo Preditivo: Random Forest Classifier (Acuracia 99.90%) para antecipacao de crises (Fase 6)
4. Sistema Multiagente: 4 agentes autônomos (OpenAI Agents SDK) + 10 protocolos de emergencia (Fase 6)

↓

CAMADA 5: APRESENTAÇÃO & EXPERIÊNCIA DO USUÁRIO (FRONT-END & MOBILE)
Web: React + Vite publicado na Vercel (CI/CD automatico via GitHub, rotas SPA vercel.json)
Mobile: React Native + Expo APK (.apk funcional via EAS Build, app.json e eas.json)
Funcionalidades: Login Supabase, laudos de ECG, painel de telemetria em tempo real e score de risco

Garantias Técnicas de Fluidez e Segurança do Fluxo:

- Comunicação Assíncrona e Baixa Latência: A telemetria opera em intervalos de 2 segundos, com tempo médio de inferência preditiva inferior a 40ms no Backend Python.
- Segurança e Isolamento: Credenciais sensíveis mantidas estritamente em variáveis de ambiente (.env), rotas autenticadas por Bearer Token e isolamento por CORS na Vercel.
- Desacoplamento Completo: Caso o hardware Wokwi esteja offline, o Web e o Mobile continuam operando normalmente via dados em cache ou simulações interativas.

1. Roteiro Estruturado do Vídeo Demonstrativo (Até 5 Minutos)

O vídeo demonstrativo comprova a integração real do ecossistema fim-a-fim, seguindo o roteiro técnico planejado de até 5 minutos com visualização das ferramentas e interfaces ativas em produção:

[Minuto 0:00 a 0:45] Abertura, Apresentação da Equipe & Visão Geral da Arquitetura:
Apresentação dos integrantes Davi Santos Ferreira e Lais Kurahashi, exibição do repositório no GitHub, apresentação do diagrama da arquitetura final de 5 camadas e esclarecimento dos objetivos do MVP integrado.

[Minuto 0:45 a 1:45] Hardware IoT e Simulação em MicroPython no Wokwi:
Demonstração do firmware ESP32 executando no Wokwi. Leitura da temperatura no DHT22 e modulação de BPM no potenciômetro. Feedback visual local com acionamento do display OLED SSD1306 e do LED vermelho de alerta. Demonstração do Buffer de Borda salvando dados offline e sincronizando automaticamente ao recuperar o sinal.

[Minuto 1:45 a 2:45] Backend Integrador Python & Motores de IA:
Demonstração dos logs do Backend Flask em execução na AWS Elastic Beanstalk. Ingestão da telemetria do ESP32 em tempo real, chamada automática ao modelo Random Forest da Fase 6 (predizendo risco e score) e execução da bateria de 7 testes automatizados de integração com 100% de sucesso.

[Minuto 2:45 a 4:00] Aplicação Web na Vercel (CI/CD) & Nova Tela de Telemetria:
Navegação na URL pública da Vercel (rotas SPA funcionais). Demonstração da tela de Análise de ECG (Fase 4), seguida da apresentação da nova tela 'Telemetria & Risco' atualizando os cards de BPM, Temperatura, status do ESP32 e diretrizes clínicas acionadas dinamicamente.

[Minuto 4:00 a 5:00] Aplicativo Mobile Android (APK via EAS Build) & Conclusão:
Demonstração do aplicativo CardioIA instalado no dispositivo Android via .apk gerado no EAS Build. Validação do fluxo de login e visualização dos indicadores cardíacos. Conclusão final da entrega da Fase 7.

2. Homologação da Solução: Testes Automatizados de Integração

A suíte de testes automatizados backend/test_integrador.py foi executada sobre os módulos integrados, validando 100% dos requisitos técnicos estipulados para a Fase 7:

```
test_01_health_check_status: Diagnóstico de saúde da API, Watson, modelos de risco e IoT -> [APROVADO]
test_02_recebimento_telemetria_micropython_normal: Ingestão de telemetria estável (BPM 76, Temp 36.6°C) -> [APROVADO]
test_03_recebimento_telemetria_micropython_alerta_critico: Ingestão de crise (BPM 138, Temp 38.6°C) -> [APROVADO]
test_04_consulta_ultimo_sinal_iot: Retorno instantâneo do último sinal para alimentação da UI -> [APROVADO]
test_05_consulta_historico_iot: Recuperação ordenada do histórico de telemetria para gráficos -> [APROVADO]
test_06_predicao_risco_personalizada: Inferência do modelo Random Forest e acionamento de protocolos -> [APROVADO]
test_07_resumo_integracao_completa: Consolidação executiva dos 4 pilares do ecossistema hospitalar -> [APROVADO]
```

3. Conclusão do MVP Final CardioIA

Com a unificação funcional dos módulos, a transição para MicroPython com feedback visual duplo (OLED/LED), o deploy Web com CI/CD na Vercel e o aplicativo Mobile .apk via Expo EAS, a equipe entrega um MVP completo, auditável e aderente às melhores práticas de engenharia de software e Inteligência Artificial aplicada à saúde.